Integración de Sistemas

Semana 8. API Gateways
y Service Mesh

Sistemas de Información — PUCE
Docente: Damián Nicolalde

Semana 8. API Gateways y Service Mesh

Objetivo: Comprender el rol de un API Gateway como punto de entrada centralizado de un ecosistema de microservicios, los patrones arquitectónicos que resuelve (BFF, Aggregator, Proxy), las tecnologías open source y comerciales más usadas (Kong, Apigee, Nginx), la distinción conceptual entre un Gateway de borde y un Service Mesh interno (Istio, Linkerd), y las políticas de seguridad y despliegue que se aplican en ese punto centralizado.

Resultado de aprendizaje asociado: Implementar un ecosistema de microservicios expuestos mediante APIs REST y gestionados con API Gateway y un bus de eventos, incorporando contratos, pruebas, observabilidad y seguridad para asegurar escalabilidad y desempeño.

Introducción

Desde la Semana 1, este curso ha construido microservicios independientes: una API REST de Productos (Semanas 1-2), un servicio gRPC (Semana 3), un servidor GraphQL (Semana 4), un servicio SOA con mensajería (Semana 5), una coreografía de eventos (Semana 6) y una disciplina de contratos formales (Semana 7). En ningún momento, sin embargo, se ha resuelto un problema que aparece inevitablemente en cuanto un sistema pasa de "un microservicio" a "un ecosistema de microservicios": ¿por dónde entra el cliente?

Si cada microservicio se expone directamente en su propio puerto, el cliente (una app móvil, un frontend web, un partner externo) tiene que conocer la dirección exacta de cada uno, implementar su propia lógica de autenticación contra cada uno, y absorber el costo de que cada microservicio cambie de forma independiente. Peor aún, cada microservicio tendría que reimplementar por separado funciones transversales — autenticación, límites de tasa, registro de auditoría — que no tienen nada que ver con su lógica de negocio particular. Esta semana resuelve ese problema con un componente arquitectónico que probablemente ya conoces de nombre: el API Gateway.

La experiencia de laboratorio de hoy es deliberadamente práctica: vas a instalar Kong Gateway en modo DB-less (sin base de datos, configurado con un solo archivo declarativo) y ponerlo delante de la API nestjs-productos-api que construiste en las Semanas 1-2, aplicando dos políticas reales — límite de tasa (rate-limiting) y propagación de un encabezado de trazabilidad — para sentir en carne propia qué gana el sistema al introducir este punto de control centralizado, y qué overhead agrega.

Conviene tener presente, antes de entrar en materia, que un API Gateway no es una tecnología nueva ni exclusiva de los microservicios: los conceptos de proxy inverso y de punto de entrada centralizado existen desde los primeros días de la web. Lo que sí es relativamente nuevo es la combinación específica de responsabilidades que un Gateway moderno asume simultáneamente —enrutamiento, seguridad, observabilidad y gobernanza— empaquetadas en un único componente configurable de forma declarativa. Esta semana estudia esa combinación con el mismo rigor con el que ya estudiaste otros componentes de infraestructura del curso: no basta con saber que "existe un Gateway", hay que entender exactamente qué problema

resuelve, cuál no resuelve, y cuándo su costo operativo se justifica frente a la alternativa de no tenerlo.

8.1. Funciones del API Gateway

El problema que resuelve: repaso de lo acumulado en 7 semanas

Piensa en el estado actual de tu propio proyecto del curso. Tienes (o podrías tener simultáneamente corriendo) nestjs-productos-api en el puerto 3000, un servicio gRPC en otro puerto, un servidor GraphQL en otro más, y potencialmente los microservicios de la Saga de la Semana 6 escuchando eventos en RabbitMQ. Si mañana quisieras construir una app móvil que consuma todo este ecosistema, tendrías que: (1) conocer y codificar en el cliente la dirección exacta de cada microservicio; (2) implementar la lógica de autenticación de forma independiente contra cada uno; (3) aplicar límites de tasa dentro de la lógica de cada microservicio por separado; (4) si un microservicio cambia de puerto o se divide en dos, actualizar el cliente móvil y republicarlo en las tiendas de aplicaciones.

Un API Gateway es un componente de infraestructura que se coloca delante de todos los microservicios, actuando como el único punto de entrada visible desde fuera del sistema. El cliente ya no habla con nestjs-productos-api directamente: habla con el Gateway, y el Gateway decide a qué microservicio interno reenviar cada petición.

Funciones típicas de un API Gateway

- Enrutamiento (routing): decide, según la ruta, el método HTTP o el contenido de la petición, a qué microservicio interno reenviarla.
- Autenticación y autorización centralizada: valida un token (JWT, API Key) una sola vez, en el borde, evitando que cada microservicio reimplemente la misma lógica.
- Rate-limiting (límite de tasa): protege a los microservicios internos de ser saturados por un cliente que envía demasiadas peticiones en poco tiempo.
- Transformación de peticiones/respuestas: agrega, quita o modifica encabezados y cuerpos, adaptando el contrato externo al interno sin tocar el código del microservicio.
- Agregación de respuestas: combina la respuesta de varios microservicios en una sola (profundizado en 8.2, patrón Aggregator).
- Caching: almacena en caché respuestas de lectura frecuente, reduciendo la carga sobre los microservicios internos.
- Observabilidad y trazabilidad: puede inyectar un correlation-id (Semana 6) en cada petición entrante.
- Versionamiento de APIs: puede enrutar /v1/productos y /v2/productos a implementaciones internas distintas.

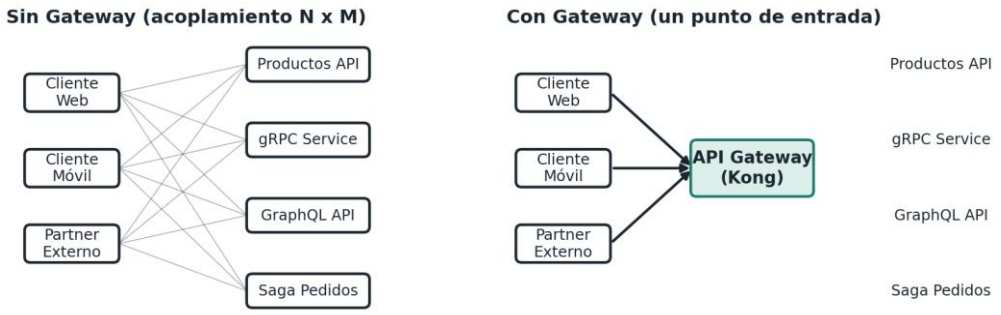
Qué NO debe hacer un Gateway

Es tentador, una vez que un equipo tiene un Gateway centralizado y potente, empezar a meter dentro de él lógica de negocio: "si el producto es de la categoría X, aplica un descuento", "si el cliente es premium, redirige a otro microservicio". Esta tentación tiene un nombre en la literatura de arquitectura: convertir el Gateway en un monolito disfrazado o God Object de la integración. Si la lógica de negocio termina viviendo en las reglas del Gateway en vez de en el código de los microservicios, se pierden exactamente las ventajas que los microservicios

prometían: despliegue independiente, propiedad clara de cada dominio, y la posibilidad de que un equipo cambie su lógica sin coordinarse con el equipo que administra el Gateway compartido.

La regla práctica que aplican los equipos experimentados es: el Gateway resuelve problemas transversales (cross-cutting concerns), nunca problemas de dominio. Autenticar un token es transversal — no depende de si el recurso es un Producto o un Cliente. Decidir si un producto tiene descuento por su categoría es una regla de negocio del dominio de Productos, y debe vivir dentro de nestjs-productos-api, no en la configuración del Gateway.

Diagrama 1. Sin Gateway: cada cliente conoce y llama directamente a cada microservicio (acoplamiento N x M). Con Gateway: un único punto de entrada centraliza el enrutamiento hacia todos los microservicios internos.



Contexto histórico: de Zuul a Kong

El concepto de API Gateway no nació con Kong. Netflix, enfrentando en 2012-2013 el mismo problema que describimos arriba pero a una escala de cientos de microservicios y millones de dispositivos cliente, construyó y liberó como open source su propio Gateway, llamado Zuul, precisamente para resolver el enrutamiento centralizado, la autenticación y el rate-limiting de todo su ecosistema. El artículo técnico que Netflix publicó explicando por qué necesitaban un "borde" (edge) programable, en vez de exponer cada microservicio directamente a Internet, se convirtió en una referencia obligatoria de la industria y anticipó casi todos los conceptos que hoy encuentras en Kong, Apigee o AWS API Gateway. Kong, fundado en 2015, tomó esta misma idea pero la construyó sobre Nginx/OpenResty y la empaquetó como un producto reutilizable por cualquier equipo — un patrón de "la gran empresa resuelve el problema primero para su propia escala, y la industria lo generaliza después" que se repite en varias otras piezas de este curso (Kafka nació en LinkedIn, gRPC en Google).

Caching en el Gateway: un ejemplo concreto

Volviendo a las funciones de la lista anterior, vale la pena detenerse en el caching, porque su valor es fácil de subestimar. Imagina que GET /productos (un catálogo que cambia con poca frecuencia) recibe 10 000 peticiones por minuto desde distintos clientes. Sin caching, cada una de esas 10 000 peticiones llega hasta nestjs-productos-api, que repite el mismo trabajo 10 000 veces para devolver, en la inmensa mayoría de los casos, exactamente la misma respuesta. Un Gateway con caching habilitado (Kong ofrece esto vía el plugin proxy-cache) puede responder desde una copia en memoria, sin siquiera despertar a nestjs-productos-api, y solo refrescar esa copia cuando expira un tiempo de vida configurado o cuando el propio microservicio invalida la caché explícitamente tras una escritura. El ahorro de carga sobre el microservicio real puede ser de varios órdenes de magnitud para endpoints de lectura de alto tráfico y baja frecuencia de cambio.

Ejercicio resuelto 8.1 — Clasificar responsabilidades

Enunciado: Clasifica cada una de estas reglas como "correcta para vivir en el Gateway" o "debe vivir dentro de un microservicio": (a) rechazar peticiones sin un token JWT válido; (b) calcular el

precio final de un producto aplicando un descuento por volumen; (c) limitar a 100 peticiones por minuto por API Key; (d) decidir si un pedido puede cancelarse según su estado actual.

Solución: (a) Gateway — es autenticación, transversal a cualquier dominio. (b) Microservicio de Productos/Pedidos — es una regla de negocio específica del dominio. (c) Gateway — rate-limiting es transversal. (d) Microservicio de Pedidos — depende del estado interno del dominio.

¿Podría un microservicio NestJS actuar como su propio Gateway simple?

Es una pregunta razonable, dado que todo el curso hasta ahora ha usado NestJS: técnicamente, sí es posible escribir un middleware o un módulo NestJS que reenvíe peticiones a otros microservicios, valide un JWT, y aplique un límite de tasa manual con un paquete como @nestjs/throttler. De hecho, el reto de la práctica de hoy (Paso 7) construye exactamente eso: un pequeño servicio NestJS, nestjs-bff-productos, actuando como un Gateway de propósito muy específico (un Aggregator).

La diferencia importante está en la escala y el propósito. Un Gateway "casero" en NestJS es apropiado para lógica específica de agregación o adaptación (BFF/Aggregator), donde el valor está precisamente en escribir código propio que combine datos de forma particular. Pero reimplementar en NestJS funciones genéricas y transversales (rate-limiting sofisticado, autenticación de decenas de rutas, un catálogo completo de plugins de seguridad) equivale, en la práctica, a reconstruir manualmente una fracción de lo que Kong ya ofrece, probado y mantenido por una comunidad externa. La regla práctica: si la necesidad es genérica y transversal, un Gateway dedicado es casi siempre más eficiente que reinventarlo; si es específica de un flujo de negocio particular, un pequeño servicio a medida en NestJS es la elección correcta.

Taller en grupo 8.1 — Auditar responsabilidades con IA

En parejas, listen 6 reglas hipotéticas de un ecosistema propio (mitad transversales, mitad de negocio, mezcladas deliberadamente). Pidan a un agente de IA: "Actúa como un arquitecto que debe decidir, para cada una de estas 6 reglas, si pertenece a un API Gateway o al código de un microservicio de dominio. Justifica cada decisión." Verificación obligatoria: para cualquier regla donde la IA dude, decidan ustedes mismos aplicando el criterio "¿es transversal, o es una regla de negocio?".

8.2. Patrones BFF, Aggregator, Proxy

Tres formas distintas de estructurar el Gateway

No todo Gateway cumple el mismo rol arquitectónico. Existen al menos tres patrones bien diferenciados, que conviene distinguir con precisión porque cada uno resuelve un problema distinto.

Proxy / Reverse Proxy simple

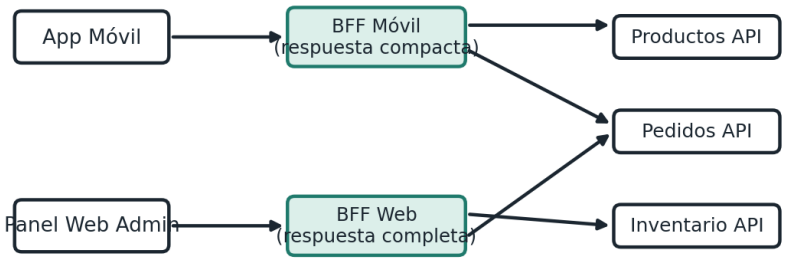
Un Proxy reenvía la petición prácticamente sin modificarla, cambiando solo la dirección de destino. Es la forma más simple de Gateway: el cliente pide /productos, el Proxy lo reenvía a nestjs-productos-api:3000/productos, y devuelve la respuesta tal cual. Kong, en su configuración más básica (la que usarás en el Paso 3 de la práctica de hoy), actúa exactamente así: define un service (el destino real) y una route (el patrón de URL público), y reenvía sin transformar nada más allá de lo que un plugin explícito agregue.

Backend-for-Frontend (BFF)

El patrón BFF reconoce una realidad práctica: distintos tipos de cliente (una app móvil, un frontend web de escritorio, un panel de administración) tienen necesidades de datos muy distintas, incluso cuando consumen el mismo dominio de negocio. Una app móvil, con ancho de banda limitado, quiere respuestas compactas con solo los campos esenciales. Un panel de administración de escritorio puede querer el objeto completo con todos sus metadatos.

En vez de forzar a un único endpoint genérico a servir a ambos (con el riesgo del overfetching que ya viste en GraphQL en la Semana 4), el patrón BFF propone construir un Gateway (o una capa delgada) por cada tipo de cliente, cada uno adaptado a sus necesidades específicas, mientras ambos consultan por debajo a los mismos microservicios de dominio.

Diagrama 2. Patrón BFF: un BFF Móvil y un BFF Web, cada uno con su propia forma de respuesta adaptada, consultando por debajo a los mismos microservicios de dominio.

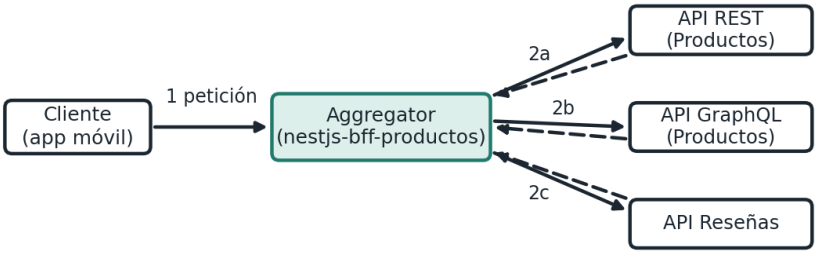


Aggregator (agregador)

El patrón Aggregator resuelve un problema distinto: cuando una sola pantalla o una sola operación del cliente necesita datos que viven en varios microservicios distintos, en vez de que el cliente haga varias llamadas por separado, el Gateway hace esas llamadas internamente y devuelve una sola respuesta combinada.

Un ejemplo concreto con el dominio del curso: imagina una pantalla de "ficha de producto" que necesita mostrar tanto los datos básicos del producto (de la API REST de las Semanas 1-2) como datos calculados vía consultas GraphQL más ricas (de la Semana 4). En vez de que el cliente móvil haga dos peticiones HTTP separadas y las combine él mismo, un Aggregator en el Gateway (o un pequeño servicio BFF dedicado) hace ambas llamadas internamente y devuelve un único JSON combinado. Esto es exactamente lo que construirás como reto en el Paso 7 de la práctica de hoy: un pequeño servicio nestjs-bff-productos que agrega datos de tu API REST y tu API GraphQL.

Diagrama 3. Patrón Aggregator: el cliente hace una sola petición al Gateway; el Gateway consulta internamente a 2-3 microservicios y combina sus respuestas en un único JSON de salida.



Origen histórico del patrón BFF

El término Backend-for-Frontend fue acuñado y popularizado alrededor de 2015 por ingenieros de SoundCloud (Phil Calçado, en un artículo ampliamente citado) que enfrentaban un problema muy concreto: su API REST "genérica" original intentaba servir simultáneamente a la app de iOS, la de Android y al reproductor web, y cada equipo de cliente terminaba pidiendo cambios incompatibles entre sí al mismo equipo de backend. La solución de SoundCloud fue dejar de tener "un backend para todos los frontends" y, en cambio, dar a cada equipo de cliente su propio backend delgado, que ellos mismos pudieran evolucionar a su propio ritmo. Esta idea — que cada equipo de frontend "posea" su propia capa BFF— es tan importante como la forma técnica de la respuesta: el patrón BFF no es solo una optimización de payload, es también una forma de organización de equipos que reduce la coordinación necesaria entre ellos.

Ejercicio resuelto 8.2b — Diagnosticar la ausencia de un BFF

Enunciado: Un equipo tiene un único endpoint GET /productos/{id}/completo que devuelve 40 campos, pensado originalmente solo para el panel de administración. Meses después, la app móvil también empieza a usarlo, y cada vez que administración agrega un campo nuevo, el equipo móvil se queja de que su app consume más datos móviles. ¿Qué patrón resolvería el problema, y qué NO resolvería?

Solución: BFF — crear un GET /productos/{id}/movil delgado, propio del equipo móvil, desacoplado del endpoint de administración. Un Aggregator no resolvería este problema, porque no hay múltiples microservicios cuyos datos combinar — hay un solo microservicio cuyo endpoint sirve, incorrectamente, a dos audiencias con necesidades opuestas.

Tabla 1. Comparación de los tres patrones

Patrón	Qué resuelve	Ejemplo del curso
Proxy simple	Ocultar la topología interna; un solo punto de entrada	Kong reenviando /productos a nestjs-productos-api
BFF	Adaptar la forma de la	Un BFF Móvil compacto vs.

	respuesta a cada tipo de cliente	un panel admin completo
Aggregator	Combinar datos de varios microservicios en una sola respuesta	nestjs-bff-productos combinando REST + GraphQL

Estos tres patrones no son mutuamente excluyentes: un sistema real suele combinar los tres. Kong Gateway puede actuar como Proxy simple para la mayoría de las rutas, mientras un servicio BFF/Aggregator dedicado maneja los pocos casos donde de verdad hace falta combinar datos de más de un microservicio.

Cómo combinan estos patrones empresas reales

Tabla 1b. Patrones dominantes en empresas reales

Empresa	Patrón(es) dominantes	Motivación documentada
Netflix	Gateway (Zuul) + múltiples BFF por dispositivo	Cientos de tipos de dispositivos con capacidades muy distintas
SoundCloud	BFF (origen histórico del patrón)	Evitar que iOS, Android y web bloquearan al mismo equipo de backend
Spotify	BFF por plataforma + Gateway interno por squad	Cientos de equipos autónomos que exponen servicios sin coordinarse
Monzo (banca digital)	API Gateway + Service Mesh interno	Alto volumen financiero con requisitos estrictos de auditoría y cifrado

Esta tabla confirma un patrón: la elección entre Gateway simple, BFF, Aggregator y Service Mesh casi nunca es una decisión puramente técnica — está fuertemente condicionada por cómo está organizada la empresa en equipos, cuántos tipos de cliente distintos debe servir, y qué tan estrictos son sus requisitos regulatorios. La arquitectura, una vez más, sigue a la organización (un eco directo de la Ley de Conway: los sistemas que una organización diseña tienden a reflejar su propia estructura de comunicación interna).

Ejercicio resuelto 8.2 — Elegir el patrón correcto

Enunciado: (a) Una app móvil necesita, en una sola pantalla, el nombre y precio de un producto (REST) y sus reseñas más recientes (microservicio de Reseñas separado). (b) Un partner externo solo necesita consultar el catálogo tal cual existe. (c) Un panel admin necesita el objeto Producto completo, mientras la app móvil solo necesita nombre, precio e imagen.

Solución: (a) Aggregator — combina datos de dos microservicios distintos. (b) Proxy simple — no hay transformación ni combinación. (c) BFF — dos formas de respuesta distintas para dos tipos de cliente distintos.

Taller en grupo 8.2 — Diseñar un BFF con IA

En equipos de 3, elijan un dominio propio con al menos dos tipos de cliente reales y describan qué campos necesitaría cada uno. Pidan a la IA: "Diseña la forma de respuesta JSON de un BFF Móvil y un BFF Web para esta entidad, dado que el móvil prioriza tamaño de respuesta y el panel web prioriza completitud de datos." Verificación obligatoria: comparen el tamaño en bytes aproximado de ambas respuestas y confirmen que la diferencia es real y justificada.

8.3. Uso de Kong, Apigee o Nginx

El panorama de gateways: open source vs. comerciales

No existe un único "API Gateway estándar" de la industria; existen varias familias de herramientas con propósitos y modelos de negocio distintos.

Tabla 2. Panorama de gateways

Gateway	Tipo	Característica distintiva
Kong Gateway	Open source (+ Enterprise)	Basado en Nginx/OpenResty, extensible vía plugins, modo DB-less declarativo
Nginx (OpenResty)	Open source	Extremadamente ligero y rápido; más configuración manual para funciones avanzadas
Apigee (Google Cloud)	Comercial/SaaS	Orientado a gobernanza a gran escala, monetización, analítica de negocio
AWS API Gateway	Comercial/gestionado	Integración nativa con AWS (Lambda, Cognito), pago por uso
Traefik	Open source	Fuerte integración con contenedores/Kubernetes, descubrimiento automático

Este curso usa Kong Gateway para la práctica de laboratorio por tres razones concretas: es completamente open source, tiene un modo de operación DB-less que elimina la necesidad de levantar una base de datos adicional solo para configurar rutas, y su arquitectura de plugins es representativa de cómo funcionan la mayoría de los gateways modernos, incluidos los comerciales.

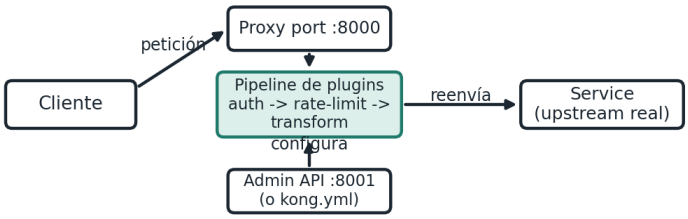
Arquitectura interna de Kong

Kong se construye sobre Nginx y OpenResty (que permite ejecutar lógica en Lua dentro de Nginx). Sus piezas esenciales son:

- Proxy port (por defecto 8000): el puerto donde Kong escucha las peticiones reales de los clientes.
- Admin API / Admin port (por defecto 8001): una API REST propia de Kong para configurar servicios, rutas y plugins. En modo DB-less, se reemplaza por un archivo declarativo (kong.yml).
- Services: representan un microservicio interno real.
- Routes: definen qué patrón de URL pública debe dirigirse a qué service.

- Plugins: unidades de funcionalidad reutilizable que se "enchufan" sobre un service o una route.

Diagrama 4. Arquitectura de Kong: una petición del cliente entra por el proxy port, atraviesa el pipeline de plugins configurados, y se reenvía al service upstream. La Admin API/kong.yml configura esta cadena.



DB-less vs. con base de datos

Kong soporta dos modos de operación. En el modo con base de datos (PostgreSQL o Cassandra), toda la configuración se almacena en una base de datos y se administra vía la Admin API en tiempo real, sin reiniciar Kong — el modo recomendado para producción a gran escala. En el modo DB-less (declarativo), toda la configuración vive en un único archivo YAML (kong.yml), que se carga al iniciar el contenedor. No requiere base de datos adicional, es reproducible (el archivo se versiona en Git), y es ideal para entornos de desarrollo, laboratorio, o despliegues donde la configuración cambia con poca frecuencia — exactamente el escenario de la práctica de hoy.

```
_format_version: "3.0"
services:
  - name: productos-service
    url: http://host.docker.internal:3000
    routes:
      - name: productos-route
        paths: ["/productos"]
    plugins:
      - name: rate-limiting
        config:
          minute: 5
          policy: local
```

Nota cómo este archivo declarativo es, en esencia, otro ejemplo de contrato schema-first (Semana 7): describe la topología deseada del sistema antes de que Kong la ejecute, de forma legible, versionable y revisable en una Pull Request, exactamente el mismo principio que ya aplicaste al JSON Schema de Producto.

Caso de estudio: por qué Kong eligió construirse sobre Nginx

Vale la pena entender una decisión de diseño que explica por qué Kong es tan popular: en vez de escribir un servidor HTTP desde cero, sus creadores decidieron construirlo encima de Nginx, uno de los servidores web más probados en producción del mundo. Esto significa que, en el núcleo de cada Gateway Kong, hay dos décadas de optimización de manejo de conexiones HTTP heredadas gratuitamente, y Kong "solo" tuvo que agregar, vía el lenguaje Lua (a través de

OpenResty), la capa de lógica de negocio del Gateway: enrutamiento dinámico, plugins, Admin API. Esta estrategia de "no reinventar la capa de infraestructura probada, construir la capa de valor diferencial encima" es un patrón de diseño de software recurrente.

Comparación directa: la misma ruta en Nginx puro vs. Kong

Para apreciar el valor agregado de Kong sobre un Nginx "pelado", compara cómo se vería configurar la misma ruta con rate-limiting en Nginx puro frente a la declaración equivalente en kong.yml:

```
# nginx.conf (fragmento) - configuracion manual, imperativa
limit_req_zone $binary_remote_addr zone=productos:10m rate=5r/m;
location /productos {
    limit_req zone=productos burst=2 nodelay;
    proxy_pass http://nestjs-productos-api:3000;
}
```

Funcionalmente equivalente al fragmento kong.yml ya visto, pero con una diferencia práctica importante: en Nginx puro, agregar un plugin de autenticación JWT, o de transformación de headers, requiere escribir o instalar módulos adicionales en C o Lua y recompilar/reconfigurar el servidor completo. En Kong, es una línea adicional bajo plugins: en el mismo archivo declarativo, reutilizando un ecosistema de más de 100 plugins ya escritos, probados y documentados por la comunidad.

Plugins típicos que usarás hoy

Tabla 3. Plugins de Kong usados en la práctica

Plugin	Qué hace	Uso en la práctica de hoy
rate-limiting	Limita el número de peticiones por unidad de tiempo	Paso 5: 5 peticiones por minuto sobre /productos
request-transformer	Agrega, quita o modifica encabezados/parámetros	Paso 6: inyectar X-Correlation-Id
key-auth	Exige una API Key válida en cada petición	Mencionado como referencia
cors	Gestiona encabezados CORS para clientes de navegador	Mencionado como referencia
jwt	Verifica y decodifica un token JWT en el borde	Mencionado como referencia (ver 8.5)
correlation-id	Genera o propaga un id único por petición	Alternativa oficial al request-transformer del Paso 6
http-log / file-log	Envía un registro estructurado a un sistema externo	Base para observabilidad centralizada (Semana 10)
response-transformer	Modifica el cuerpo o encabezados de la respuesta	Ocultar campos sensibles sin tocar el microservicio

Vale la pena notar que casi todos estos plugins pueden combinarse libremente sobre la misma route, formando exactamente el pipeline que muestra el Diagrama 4: una petición real puede atravesar jwt, luego rate-limiting, luego correlation-id y finalmente response-transformer, todo sin que el microservicio de destino sepa que ninguno de estos pasos ocurrió.

Ejercicio resuelto 8.3 — Leer un kong.yml ajeno

Enunciado: Sin ver ningún otro archivo, responde: (a) ¿a qué URL interna reenvía Kong las peticiones a /pedidos?, (b) ¿cuántas peticiones por minuto se permiten?

```
services:
  - name: pedidos-service
    url: http://pedidos-interno:4000
    routes:
      - name: pedidos-route
        paths: ["/pedidos"]
    plugins:
      - name: rate-limiting
        config: { minute: 30 }
```

Solución: (a) http://pedidos-interno:4000 — el campo url del service es el destino real (upstream). (b) 30 peticiones por minuto — definido en config.minute del plugin rate-limiting.

Taller en grupo 8.3 — Diseñar un kong.yml con IA

En parejas, describan a la IA dos microservicios propios y pídanle: "Genera un archivo kong.yml en modo DB-less que enrute estos dos servicios, con el plugin rate-limiting aplicado a ambos con límites distintos, justifica por qué." Verificación obligatoria: validen manualmente la sintaxis YAML generada contra la documentación oficial de Kong antes de darla por buena.

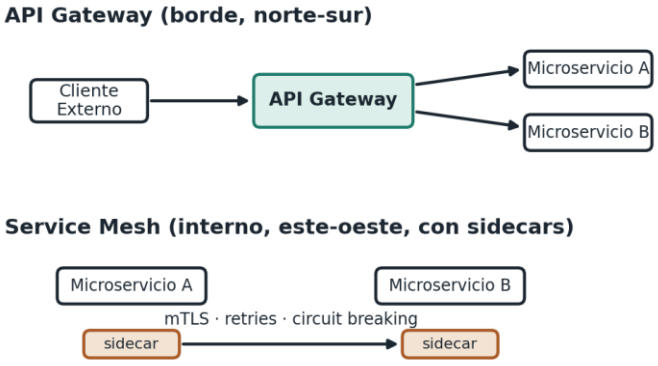
8.4. Introducción a Istio y Linkerd

Norte-sur vs. este-oeste: dos tipos de tráfico distintos

Hasta ahora, esta sección ha hablado de tráfico norte-sur: el que entra desde fuera del sistema hacia el borde de la arquitectura. Pero existe otro tipo de tráfico, igual de importante, que un API Gateway de borde normalmente no gestiona: el tráfico este-oeste, es decir, la comunicación entre microservicios internos — por ejemplo, cuando PedidoService llama internamente a InventarioService (Semana 6), sin que ninguna petición externa esté involucrada en ese momento.

Un Service Mesh (malla de servicios) es una capa de infraestructura dedicada a gestionar precisamente ese tráfico este-oeste, con las mismas preocupaciones (seguridad, observabilidad, resiliencia) que un Gateway aplica al tráfico norte-sur, pero aplicadas a las llamadas internas entre microservicios.

Diagrama 5. API Gateway (borde, tráfico norte-sur) vs. Service Mesh (interno, tráfico este-oeste, mediante sidecars adjuntos a cada microservicio).



Cómo funciona un Service Mesh: el patrón sidecar

La forma más común de implementar un Service Mesh (usada tanto por Istio como por Linkerd) es el patrón sidecar: junto a cada instancia de un microservicio, se despliega un segundo proceso ligero (normalmente un proxy como Envoy) que intercepta todo el tráfico de red que entra y sale de ese microservicio. El microservicio ni siquiera necesita saber que el sidecar existe: sigue haciendo llamadas HTTP normales a otros microservicios, pero esas llamadas en realidad pasan primero por su propio sidecar y luego por el sidecar del destino.

Este diseño permite que el Service Mesh resuelva, de forma completamente transparente para el código de negocio, problemas como:

- mTLS automático (mutual TLS): cifrado y autenticación mutua entre cada par de microservicios, sin que ninguno implemente TLS en su propio código.
- Reintentos y timeouts configurables centralmente, de forma uniforme para todo el mesh.
- Circuit breaking: el sidecar puede dejar de enviar tráfico a una instancia que está fallando repetidamente (retomado en la Semana 15).
- Observabilidad interna granular: métricas de latencia, error y volumen entre cada par específico de microservicios.

Envoy: el proxy que hizo posible el Service Mesh moderno

Detrás de Istio (y de muchos otros sistemas) está Envoy, un proxy de alto rendimiento escrito en C++ y originalmente desarrollado por Lyft, liberado como open source en 2016. Envoy fue diseñado desde el inicio pensando en observabilidad de primera clase, soporte nativo de HTTP/2 y gRPC (Semana 3), y una API de configuración dinámica que permite reconfigurar el proxy en caliente, sin reiniciarlo — esencial cuando hay miles de sidecars que necesitan actualizar sus reglas constantemente. Istio, en esencia, es un plano de control que genera y distribuye automáticamente la configuración de miles de proxies Envoy (el plano de datos) para que actúen de forma coordinada como un solo Service Mesh coherente.

Un vistazo a la configuración declarativa de Istio

Aunque este curso no incluye una práctica hands-on con Istio (por su dependencia de Kubernetes), vale la pena ver brevemente su configuración, porque revela un patrón familiar: al igual que kong.yml, Istio también configura el comportamiento de forma declarativa.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: productos-service
spec:
  hosts: ["productos.interno"]
  http:
    - route:
        - destination: { host: productos-service, subset: v1 }
          weight: 90
        - destination: { host: productos-service, subset: v2 }
          weight: 10
```

Este fragmento -conceptualmente el mismo despliegue canary que se verá aplicado al Gateway en la sección 8.5, pero aplicado aquí al tráfico interno este-oeste- reparte el 90% del tráfico interno hacia productos-service v1 y el 10% hacia v2. El mismo patrón de despliegue progresivo puede aplicarse tanto en el borde (Gateway) como en el interior (Service Mesh) del sistema; lo que cambia es la capa de infraestructura responsable de ejecutarlo.

Istio y Linkerd: dos implementaciones con filosofías distintas

Tabla 4. Istio vs. Linkerd

Aspecto	Istio	Linkerd
Origen	Google, IBM, Lyft (basado en Envoy)	Buoyant (proxy propio en Rust)
Complejidad de operación	Alta — muchas opciones	Deliberadamente más simple
Curva de adopción	Más pronunciada	Más suave
Casos de uso típicos	Enrutamiento avanzado de tráfico (canary, mirroring)	Simplicidad operativa
Requiere	Kubernetes	Kubernetes

Cuándo SÍ y cuándo NO conviene un Service Mesh

Ambas herramientas comparten un requisito fundamental: ambas requieren Kubernetes como plataforma de orquestación de contenedores subyacente — por esta razón, este curso las trata de forma conceptual, sin una práctica hands-on (a diferencia del Gateway de la sección 8.3, que sí se implementa hoy con Docker directamente).

Con 2-5 microservicios, la complejidad operativa de instalar y mantener un Service Mesh casi siempre supera el beneficio. Con decenas o cientos de microservicios, gestionados por equipos distintos, la alternativa a un Service Mesh es que cada equipo reimplemente su propia lógica de reintentos, timeouts y cifrado de forma inconsistente — y ahí es donde un Service Mesh

empieza a pagar dividendos reales. Esta es la misma pregunta de "¿cuándo se justifica la complejidad adicional?" que ya te hiciste en la Semana 5 al decidir cuándo introducir un ESB, y que volverás a ver en la Semana 13 (ESB vs. Microservicios).

Tabla 3b. Señales para decidir si conviene un Service Mesh

Señal en tu ecosistema	¿Justifica un Service Mesh?
2-5 microservicios, un solo equipo	No — la complejidad operativa supera el beneficio
Decenas de microservicios, varios equipos, sin mTLS interno	Empieza a justificarse, especialmente con requisitos de auditoría
Cientos de microservicios, alta rotación de versiones, canary interno fino	Sí — el escenario para el que Istio fue diseñado
Ningún equipo con experiencia previa en Kubernetes	Señal de alerta: la curva de aprendizaje combinada puede costar más que el problema

Ejercicio resuelto 8.4 — Gateway vs. Service Mesh

Enunciado: Clasifica cada necesidad: (a) un cliente móvil externo necesita autenticarse con JWT; (b) PedidoService necesita reintentar una llamada fallida a InventarioService; (c) se necesita cifrado en tránsito entre todos los microservicios internos; (d) se necesita limitar cuántas peticiones por segundo puede enviar un partner externo.

Solución: (a) Gateway — tráfico norte-sur. (b) Service Mesh — tráfico este-oeste. (c) Service Mesh — mTLS interno. (d) Gateway — tráfico norte-sur de un consumidor externo.

Taller en grupo 8.4 — Debate: ¿nuestro proyecto necesita un Service Mesh?

En equipos de 4, tomando el ecosistema de microservicios construido durante el curso, debatan con la IA como moderador: "Actúa como un arquitecto escéptico. Dado este ecosistema de N microservicios, argumenta en contra de introducir un Service Mesh ahora, y luego a favor, si creciéramos a 50 microservicios." Verificación obligatoria: el equipo debe llegar a una conclusión propia y explícita, no limitarse a repetir la respuesta de la IA.

8.5. Políticas de seguridad y despliegue

Políticas de seguridad aplicadas en el borde

Concentrar el tráfico externo en un único punto de entrada tiene una ventaja de seguridad evidente: es mucho más fácil auditar, actualizar y asegurar un solo componente que N componentes distintos. Las políticas de seguridad típicas aplicadas en ese punto centralizado incluyen:

- Rate-limiting / throttling: primera línea de defensa contra ataques de denegación de servicio (DoS) y scraping abusivo.
- IP whitelisting/blacklisting: restringir el acceso a rangos de direcciones IP conocidos.
- Autenticación de API Keys o JWT en el borde: validar la identidad del consumidor antes de que llegue a cualquier microservicio interno.
- Validación de esquema en el borde: algunos gateways pueden validar el cuerpo de una petición contra un JSON Schema (Semana 7) antes de reenviarla.

JWT vs. API Key en el borde: ¿cuál usar?

Tabla 4b. API Key vs. JWT

Aspecto	API Key	JWT
¿Qué identifica?	Una aplicación/consumidor	Puede identificar a una persona autenticada, con claims
¿Contiene información propia?	No — identificador opaco	Sí — id de usuario, roles, expiración embebidos
Revocación	Inmediata (se desactiva la key)	Difícil antes de expirar, salvo lista de revocación
Caso de uso en el curso	Partners B2B del catálogo de Productos	Usuario final en el Reto de Pedidos (Semana 6)

La elección no es excluyente: un Gateway real frecuentemente combina ambos — API Keys para identificar y limitar la tasa de aplicaciones externas, JWT para autenticar y autorizar a usuarios finales dentro de esas mismas aplicaciones.

El estándar OWASP API Security Top 10

Tabla 4c. Riesgos OWASP API Security mitigados en el borde

Riesgo (abreviado)	Cómo ayuda el Gateway
Autorización a nivel de objeto rota	El Gateway valida el token, pero la autorización fina por recurso sigue siendo del microservicio
Autenticación rota	Centralizar la validación en un solo punto reduce implementaciones inconsistentes
Consumo de recursos sin límite	Mitigado por el plugin rate-limiting (Paso 5 de

	la práctica)
Exposición excesiva de datos	Relacionado con BFF (8.2): responder solo lo necesario reduce la superficie expuesta
Configuración de seguridad incorrecta	Un kong.yml versionado y revisado en PRs reduce configuraciones inconsistentes

El Gateway mitiga varios riesgos de seguridad transversales, pero no todos — la autorización fina por recurso, en particular, casi siempre debe verificarse también dentro del microservicio de dominio, porque solo él conoce las reglas de negocio necesarias para decidir correctamente.

El concepto de Zero Trust aplicado a APIs

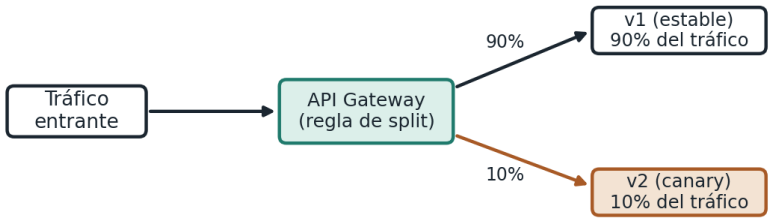
Un principio de seguridad cada vez más adoptado, tanto en el borde como en el interior, es Zero Trust: la idea de que ninguna petición debe considerarse confiable solo por su origen, sino que cada petición, sin importar de dónde venga, debe autenticarse y autorizarse explícitamente. Este principio explica por qué un Service Mesh aplica mTLS incluso a la comunicación interna entre microservicios (sección 8.4): bajo Zero Trust, "interno" ya no es sinónimo de "confiable por defecto".

Estrategias de despliegue a nivel de rutas del Gateway

Un Gateway centralizado no solo gestiona seguridad: también es el lugar natural para implementar estrategias de despliegue progresivo, sin necesidad de coordinar cambios de DNS o de infraestructura de red.

- Despliegue canary: una nueva versión (v2) recibe solo un pequeño porcentaje del tráfico real, mientras la mayoría sigue yendo a la versión estable (v1). Si las métricas de v2 son saludables, el porcentaje se incrementa gradualmente.
- Despliegue blue-green: existen dos entornos completos corriendo en paralelo; el Gateway cambia el 100% del tráfico de uno al otro de forma instantánea, y puede revertir igual de rápido.

Diagrama 6. Despliegue canary a nivel de rutas: el Gateway divide el tráfico entrante entre dos versiones internas del microservicio (90% a v1, 10% a v2).



Gobernanza de APIs

Más allá de la seguridad técnica, un Gateway centralizado se convierte, en organizaciones con muchos equipos, en el lugar natural para ejercer gobernanza de APIs: mantener un catálogo actualizado de qué rutas existen, quién es dueño de cada una, qué versión está vigente y cuál está en proceso de deprecación (Semana 2 y Semana 7), y qué políticas de seguridad mínimas debe cumplir cualquier ruta nueva antes de publicarse. Sin esta gobernanza, un ecosistema de microservicios tiende, con el tiempo, a acumular rutas huérfanas, versiones olvidadas y políticas de seguridad inconsistentes — el mismo tipo de deuda técnica silenciosa que ya viste, en otra forma, al hablar de contratos sin versionar en la Semana 7.

Ejercicio resuelto 8.5 — Diseñar una política de rate-limiting razonable

Enunciado: Tu API de Productos tiene dos tipos de consumidores: un frontend web propio (tráfico interno, confiable) y partners externos vía API Key pública. ¿Aplicarías el mismo límite de tasa a ambos?

Solución: No necesariamente. Es común aplicar un límite más generoso al tráfico interno confiable, identificado por su origen de red o una API Key interna dedicada, y un límite más estricto al tráfico de partners externos, cuyo comportamiento no se controla directamente. Kong permite esto configurando rate-limiting con distintos límites por consumer en vez de un único límite global.

Ejercicio resuelto 8.5b — Diseñar la expiración y renovación de un JWT en el borde

Enunciado: Tu equipo debe decidir el TTL de un JWT para usuarios finales de la app móvil del Reto de Pedidos (Semana 6). Un TTL largo (30 días) reduce reautenticaciones pero aumenta el riesgo si se filtra; un TTL corto (5 minutos) es más seguro pero degrada la experiencia. ¿Cómo resuelve la industria esta tensión?

Solución: Usar dos tokens con vidas distintas: un access token de vida corta (minutos u horas), que es el que viaja en cada petición y el que el Gateway valida en el borde, y un refresh token de vida larga, que el cliente usa sin intervención del usuario para obtener un access token nuevo cuando el anterior expira. Esto da lo mejor de ambos mundos: ventana de exposición corta y

experiencia fluida. El Gateway solo valida la firma y expiración del access token — nunca ve ni gestiona el refresh token.

Taller en grupo 8.5 — Diseñar la seguridad del borde con IA

En equipos de 3, describan su propio ecosistema y pidan a la IA: "Actúa como un arquitecto de seguridad. Propón qué políticas aplicarías en el borde (Gateway) y cuáles NO tendría sentido aplicar ahí sino dentro de cada microservicio, con tu razonamiento." Verificación obligatoria: contrasten la respuesta contra el criterio de 8.1 — cualquier política que la IA proponga para el Gateway pero que dependa del dominio de negocio debe reclasificarse.

Errores comunes al operar un API Gateway

- Meter lógica de negocio en el Gateway "porque es más rápido" que modificar el microservicio correspondiente.
- No versionar el archivo de configuración del Gateway en un repositorio de control de versiones.
- Aplicar el mismo límite de rate-limiting a todos los consumidores por igual, sin distinguir tráfico interno de externo.
- Confundir "tenemos un Gateway" con "estamos seguros" — la autorización fina por recurso sigue siendo de cada microservicio.
- Introducir un Service Mesh completo para un ecosistema de 3 microservicios, pagando una complejidad que no se necesita a esa escala.
- No monitorear el propio Gateway como componente crítico: si cae, todo el sistema queda inaccesible desde fuera, sin importar que cada microservicio interno funcione.

Caso de estudio integrador: modernizando el acceso externo de un ecosistema heredado

Cerramos la semana con un caso que integra las cinco secciones anteriores. Una empresa mediana tiene 6 microservicios construidos en distintos momentos por distintos equipos, cada uno expuesto directamente en su propio puerto público, cada uno con su propia lógica de autenticación inconsistente. Los clientes externos —una app móvil y tres partners B2B— se quejan de que integrarse con la empresa es confuso. El equipo introduce un API Gateway siguiendo cuatro pasos que resumen el contenido de esta semana:

1. Inventariar y clasificar el tráfico existente (8.1): identificar qué llamadas son transversales y cuáles son lógica de negocio específica, sin tocar ninguna lógica de negocio en este paso.

2. Diseñar el patrón de acceso correcto por tipo de cliente (8.2): la app móvil recibe un BFF Móvil delgado; los partners B2B se enrutan mediante Proxy simple directo.

3. Centralizar autenticación y rate-limiting en Kong (8.3): cada microservicio deja de implementar su propia autenticación; Kong valida un JWT único en el borde y aplica límites diferenciados por tipo de consumidor.

4. Dejar explícitamente fuera de alcance un Service Mesh (8.4): con solo 6 microservicios, el equipo decide, de forma consciente y documentada, que Istio o Linkerd sería sobre-ingeniería en este momento.

Nótese que, igual que en el caso de estudio de la Semana 7, en ningún momento se reescribió la lógica de negocio de ninguno de los 6 microservicios: el valor completo de esta modernización proviene de centralizar responsabilidades transversales duplicadas e inconsistentes, no de tocar el código de dominio.

Resumen de la semana

Tabla 5. Resumen por sección

Sección	Idea central
8.1	Un API Gateway centraliza responsabilidades transversales; nunca debe absorber lógica de negocio.
8.2	BFF adapta la respuesta a cada tipo de cliente; Aggregator combina datos de varios microservicios; Proxy simple solo reenvía.
8.3	Kong (open source, DB-less) enruta mediante services, routes y plugins, en un kong.yml versionable.
8.4	El Gateway gestiona tráfico norte-sur; un Service Mesh (Istio/Linkerd, con sidecars) gestiona tráfico este-oeste.
8.5	El Gateway centraliza seguridad y estrategias de despliegue progresivo (canary, blue-green) a nivel de rutas.

Preguntas frecuentes

¿Un API Gateway reemplaza al balanceador de carga? No necesariamente — muchos gateways también hacen balanceo de carga, pero un balanceador puro no tiene autenticación, rate-limiting ni transformación.

¿Necesito Kong para un proyecto pequeño? Depende del número de microservicios y consumidores externos. Con 3 o más microservicios y clientes externos, el Gateway empieza a pagar dividendos rápidamente.

¿Puedo usar Kong sin Docker? Sí, existen instalaciones nativas y para Kubernetes (Kong Ingress Controller). La práctica usa Docker por simplicidad.

¿Istio y Linkerd compiten con Kong? No directamente — resuelven capas distintas (borde vs. interno). Es común usar ambos simultáneamente si la escala lo justifica.

¿Qué pasa si el Gateway mismo se cae? Es, por diseño, un punto único de fallo para el tráfico externo. En producción esto se mitiga desplegando múltiples instancias del Gateway detrás de un balanceador de carga propio.

¿El Gateway agrega latencia perceptible? Técnicamente sí, un salto de red adicional, pero suele medirse en pocos milisegundos, compensado con creces por caching, rate-limiting temprano y menor necesidad de lógica repetida en cada servicio.

Recursos de profundización

- Explorar el modo con base de datos de Kong (PostgreSQL) y su Admin API en tiempo real, comparándolo contra el modo DB-less usado hoy.
- Instalar minikube o kind (Kubernetes local) y seguir el tutorial oficial de Istio ("Getting Started") para ver un Service Mesh real funcionando.
- Investigar el plugin oauth2 de Kong como alternativa más completa al jwt simple, para flujos de autorización delegada.
- Leer el artículo original de Netflix sobre Zuul y compararlo con la arquitectura de Kong usada hoy.

Glosario de términos

- API Gateway: componente centralizado que actúa como único punto de entrada hacia un ecosistema de microservicios.
- Service Mesh: capa de infraestructura, basada en sidecars, que gestiona la comunicación interna entre microservicios.
- BFF (Backend-for-Frontend): un backend delgado, adaptado a las necesidades de un tipo específico de cliente.
- Aggregator: patrón que combina respuestas de varios microservicios en una sola respuesta.
- Sidecar: proceso auxiliar desplegado junto a cada instancia de un microservicio, que intercepta su tráfico de red.
- mTLS (mutual TLS): cifrado y autenticación mutua entre dos partes de una comunicación.
- Rate-limiting: mecanismo que limita el número de peticiones permitidas en una ventana de tiempo.
- Norte-sur / Este-oeste: tráfico externo hacia el borde / tráfico interno entre microservicios.
- DB-less (Kong): modo de operación con toda la configuración en un archivo declarativo, sin base de datos.
- Envoy: proxy de alto rendimiento (C++, Lyft, 2016) usado como plano de datos por Istio y otros sistemas de Service Mesh.
- Zero Trust: principio de seguridad según el cual ninguna petición se considera confiable por su origen.
- Despliegue canary: una nueva versión recibe solo un porcentaje pequeño del tráfico real, incrementado gradualmente.
- Single point of failure: componente cuya caída interrumpe todo el sistema que depende de él, sin redundancia.

Referencias

- Kong Inc. (2024). Kong Gateway Documentation. https://docs.konghq.com
- Nginx Inc. (2024). Nginx Documentation. https://nginx.org/en/docs/
- Istio Authors. (2024). Istio Documentation. https://istio.io/latest/docs/
- Buoyant. (2024). Linkerd Documentation. https://linkerd.io/2/overview/
- Richardson, C. (2023). Pattern: Backends for Frontends. microservices.io

- Google Cloud. (2024). Apigee Documentation. https://cloud.google.com/apigee/docs

Autoevaluación

1. Explica con tus palabras por qué meter una regla de negocio dentro de la configuración del Gateway se considera un antipatrón, aunque técnicamente sea posible.

2. Da un ejemplo propio (no del capítulo) de un escenario donde convenga un patrón Aggregator en vez de dejar que el cliente haga varias llamadas por separado.

3. ¿Por qué el modo DB-less de Kong es especialmente apropiado para un entorno de laboratorio, y en qué escenario preferirías el modo con base de datos?

4. Explica la diferencia entre tráfico norte-sur y este-oeste con un ejemplo del ecosistema de microservicios de este curso.

5. ¿Por qué Istio y Linkerd requieren Kubernetes? ¿Qué problema resuelve Kubernetes que hace posible el patrón sidecar a esa escala?

6. Diseña un escenario donde aplicarías un despliegue canary a nivel de rutas del Gateway, y explica qué señal usarías para decidir aumentar el tráfico hacia la versión nueva.

7. Compara JWT y API Key como mecanismos de identificación en el borde: ¿en qué escenario del ecosistema de este curso usarías cada uno, y por qué la revocación es más sencilla con uno que con el otro?